High-Level Design (HLD) Document

Adaptive Offline-First Intelligent Data Pre-Fetching System (AOFIDPS)

For Low Connectivity Regions

Table of Contents

1. Executive Summary

Problem Statement

Low-connectivity and no-signal regions suffer from digital exclusion. Users cannot reliably access:

- Educational resources

- Healthcare guidance
- Emergency communication
- Government and financial services

Continuous internet dependency renders modern digital services unusable in rural, disaster-prone, and infrastructure-deficient environments.

Proposed System

The Adaptive Offline-First Intelligent Data Pre-Fetching System (AOFIDPS) is a modular, distributed mobile platform that:

- Predicts user needs using Edge AI.
- Pre-fetches and aggressively compresses prioritized content.
- Enables full, read-write offline functionality.
- Synchronizes intelligently and idempotently upon reconnection.
- Provides opportunistic emergency communication via peer-to-peer ad-hoc networking.

This system is built on edge intelligence, distributed networking, and cloud-native backend architecture, shifting the compute and storage paradigm directly to the user's device.

2. System Overview

2.1 Project Background

Existing mobile applications inherently assume stable, continuous connectivity. Offline support in standard apps is typically limited to read-only cached data and is non-intelligent. Rural and remote populations require predictive caching, autonomous local decision engines, opportunistic relay communication, and low-bandwidth optimized synchronization.

2.2 Vision

Create a hyper-resilient digital ecosystem that operates effectively and autonomously in intermittent and zero-connectivity environments, ensuring fundamental human rights to information, health, and safety are met regardless of telecom infrastructure.

2.3 Key Objectives

- **Reduce internet dependency by > 70%** for daily tasks.
- **Ensure emergency message latency < 5 seconds** when a relay gateway is available.
- **Enable fully offline education and healthcare decision support** using localized AI inference.
- **Provide staged digital transactions** that guarantee exact-once execution upon sync.

2.4 Scope and Boundaries

In Scope:

- On-device AI prediction (Edge ML).
- Intelligent Prefetch engine with dynamic storage management.
- Secure delta synchronization.
- Emergency opportunistic relay (BLE/WiFi Direct).
- Modular service architecture.

Out of Scope:

- Satellite hardware integration (relying strictly on standard consumer mobile hardware).
- Replacing telecom infrastructure (acting as an overlay network, not an ISP).

3. Architecture Principles

3.1 Design Philosophy

- **Offline-First:** The system assumes offline is the default state; internet is treated as a temporary enhancement.
- **Edge Intelligence:** Inference, prediction, and decision-making happen locally.
- **Privacy-by-Design:** Zero-knowledge architectures for relays and anonymized data pipelines.
- **Event-Driven Architecture:** Asynchronous queuing for all write operations.
- **Modular Extensibility:** Independent modules that can be updated or swapped without rebuilding the core engine.

3.2 Architectural Patterns

- **Edge Computing:** Pushing ML models to the client.
- **Microservices Backend:** Scalable, independent cloud services.
- **CQRS (Command Query Responsibility Segregation):** Separating local read models from deferred write commands.
- **Event Streaming:** For conflict resolution and data synchronization.
- **Opportunistic Mesh Networking:** Store-and-forward peer-to-peer data propagation.

3.3 Critical Design Decisions

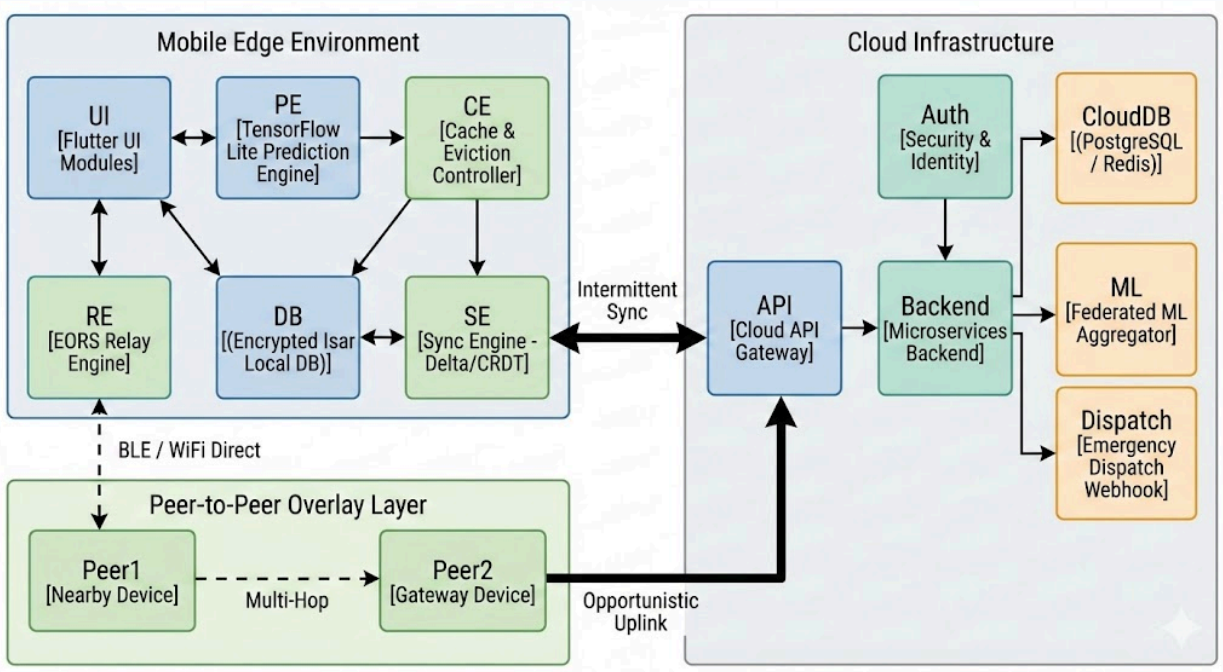
- **On-device ML inference:** To guarantee absolute privacy and zero-latency decision support.
- **BLE & WiFi Direct for discovery:** Balances battery life (BLE) with payload capacity (WiFi Direct).
- **Delta sync only:** Syncing only state changes (diffs), never full objects, to conserve precious bandwidth.
- **Encrypted local storage:** AES-256 for all at-rest data to prevent physical extraction.
- **Priority-based data eviction (LRU + Semantic Weight):** Deleting locally cached data based on both Least Recently Used metrics and contextual unimportance.

4. Technology Stack

- **Mobile:** Flutter, Dart, Isar DB (ACID compliant NoSQL for Edge), TensorFlow Lite / TFLite Micro, Native BLE/WiFi Direct APIs.
- **Backend:** FastAPI (Python) for ML endpoints, Node.js for I/O routing, PostgreSQL (Relational), Redis (Caching), Cloud Run (Serverless compute), S3-compatible Object Storage.
- **Security:** TLS 1.3, ECC (Elliptic Curve Cryptography) Key Pairs, AES-256, JWT, Signal Protocol (for E2E messaging).

5. System Architecture

5.1



5.2 Core Platform Components

1. **Prediction Engine:** Analyzes usage behavior, contextual data (location, time), and peer trends to predict what the user will need next.
2. **Prefetch Manager:** Triggers when a high-quality connection is detected to silently download predicted data blocks.
3. **Cache Controller:** Enforces storage quotas. Uses semantic eviction (e.g., deleting a completed 1st-grade math video before a pending medical article).
4. **Sync Engine:** Manages queued actions (Commands) and fetches updates (Queries) using CRDTs (Conflict-free Replicated Data Types) to prevent version collisions.
5. **Relay Engine (EORS):** Manages background BLE advertising and scanning for opportunistic routing.
6. **Security Manager:** Handles key generation, payload encryption, and certificate pinning.
7. **Cloud API Gateway:** The singular, rate-limited ingress point for all mobile traffic.

6. Module Architecture Overview

This section breaks down the 4 core modules, applying extreme depth to the problem-solution logic, architectural execution, and situational data flows.

6.1 Educational Content Module

Core Problem

Students face fragmented learning. When networks drop, video streams fail, progress tracking stops, and learning materials become inaccessible, breaking the pedagogical cycle.

Deep Solution

AI-driven predictive content pre-fetching mapped strictly to curriculum dependency graphs, combined with modular micro-packaging of content.

Architecture

- **Curriculum Dependency Graph:** A DAG (Directed Acyclic Graph) maps learning nodes. You cannot fetch Node C without caching Node A and B.
- **Behavioral Modeling (Edge ML):** Tracks learning velocity. If a student completes 1 module per day, the system pre-fetches 3 days' worth. If 5 per day, it pre-fetches 15 days' worth.
- **Delta Updates:** Quiz results and timestamps are synced via tiny JSON diffs (bytes), not by re-uploading the user profile.
- **LRU + TTL Eviction:** Completed modules degrade in storage priority over a 7-day Time-To-Live, eventually being deleted.

Situational Implementation Approach

- *Situation:* A student is taking a high-school physics course and loses internet for 3 weeks due to monsoon damage.
- *Execution:* The system already noted the student's enrollment and local region. While they had internet, the *Prefetch Manager* downloaded the next 4 chapters. Text is stored natively, images are converted to WebP (80% compression), and videos are transcoded locally to H.265 480p blocks.
- *Offline Capabilities:* The student watches the videos, takes the local auto-graded quizzes (logic executed in Dart, not a server call), and the *Sync Engine* queues the 'Progress State' events to a local CRDT log, waiting to push to the cloud seamlessly upon future reconnection.

6.2 Healthcare Resources Module

Core Problem

Remote communities lack real-time access to doctors or medical databases. Misdiagnosis in isolated regions leads to high mortality due to an inability to query symptoms against modern

medical knowledge.

Deep Solution

An on-device, localized Medical Inference Engine capable of executing complex WHO protocol decision trees, combined with an epidemiological forecasting pre-fetcher.

Architecture

- **Decision Tree Inference Engine:** A localized rules engine (e.g., converting clinical guidelines into binary search trees) ensuring sub-millisecond offline diagnosis generation.
- **Symptom Input Module:** NLP-based or structured questionnaire that feeds data into the localized TFLite model.
- **Epidemiological Pre-fetching:** Backend analytics monitor regional disease vectors. If Malaria spikes 50 miles away, the backend forces an urgent pre-fetch of Malaria protocols to all devices in the surrounding 100-mile radius *before* the network fails.
- **Encrypted Sync Logs:** Patient queries are logged and heavily anonymized (k-anonymity) before syncing to the cloud for regional health monitoring.

Situational Implementation Approach

- *Situation:* A community health worker in a dead zone encounters a patient with a severe, unknown rash and fever.
- *Execution:* The worker inputs symptoms into the offline app. The *Inference Engine* traverses the localized WHO decision tree. It rules out common colds and identifies a high probability of Dengue Fever. It immediately surfaces the locally cached treatment protocol (hydration, paracetamol, avoidance of NSAIDs).
- *Security/Safety:* To prevent AI hallucinations, the module utilizes strict protocol-based logic rather than generative AI. The diagnosis is tagged as "Algorithmic Suggestion" requiring human verification, legally protecting the system.

6.3 Communication Tools Module (EORS Integrated)

Core Problem

In absolute network failure (disasters, deep rural areas), traditional communication collapses. Lives are lost because an emergency signal cannot travel the 5 miles to the nearest active cell tower.

Deep Solution

The Emergency Opportunistic Relay System (EORS). A zero-configuration, encrypted, multi-hop mesh network leveraging ambient nearby devices to securely transport packets to a gateway.

Architecture

- **BLE Discovery & Handshake:** Devices constantly emit low-energy beacons. When two devices intersect, they establish a rapid, silent connection.
- **Multi-hop Routing (Store & Forward):** Device A (Victim) -> Device B (Passerby) -> Device C (Bus Driver with Wi-Fi).
- **Encrypted Payload Forwarding:** Uses Public Key Infrastructure (PKI). The payload is encrypted with the *Backend Dispatch's* public key. Intermediary nodes (Devices B and C) act strictly as blind mules; they cannot read the medical or location data.
- **Duplicate Suppression (UUID):** Prevents infinite network flooding. If Device B has already forwarded Packet ID #9981, it drops any subsequent identical packets.

Situational Implementation Approach

- *Situation:* A vehicle crashes in a cellular dead zone. The driver triggers an SOS.
- *Execution:*
    1. The app generates a 512-byte packet: [Timestamp] + [GPS Coords] + [Medical ID] + [Hash].
    2. The packet is encrypted.
    3. The device overrides standard BLE limits to broadcast at maximum power (High-Duty Cycle).
    4. Another car drives past. Its AOFIDPS app silently receives the encrypted packet and stores it in the *Isar DB*.
    5. Twenty minutes later, the passing car enters a 4G zone. The *Sync Engine* detects connectivity.
    6. *Priority Interrupt:* The Sync Engine pauses downloading educational videos and instantly pushes the EORS packet to the Cloud API.
    7. The Backend decrypts the payload and triggers an API webhook to local EMS dispatch.

6.4 Essential Digital Services Module

Core Problem

Economic and civic participation requires digital access. Users cannot complete government subsidies, micro-loans, or supply-chain logs without timing out online forms.

Deep Solution

Transactional staging with cryptographically secure, delayed execution using event sourcing and idempotent APIs.

Architecture

- **Offline Form Builder:** Complex forms are rendered via JSON schemas cached locally. Field validation happens offline.
- **Local Transaction Tokenization:** Once a form is submitted offline, the system generates a signed JWT token representing the "Intent to Execute".

- **Idempotent Sync Protocol:** Ensures that when the network restores, if the request is sent twice (due to packet drops), the backend only processes it exactly once.
- **Conflict-Free Replicated Data Types (CRDTs):** If a user updates their farm inventory offline, and a cooperative manager updates it online, CRDT logic deterministically merges the edits without manual conflict resolution.

Situational Implementation Approach

- *Situation:* A farmer needs to log their harvest yield for government subsidy payment but is in a field with no signal.
- *Execution:* The farmer opens the app, fills out the form, and hits "Submit". The UI shows "Submitted - Queued for Sync". The *Event Streaming* architecture writes a Command to the local queue.
- *Resolution:* The farmer returns home to a Wi-Fi connection. The Sync Engine fires. It reads the Command queue and sends the payload to the backend. The backend processes the idempotency key. If successful, it sends a Webhook back updating the local UI to "Approved & Processed".

7. Data Architecture

Mobile (Edge) Data Structure

- **Encrypted Isar DB:** Fast, asynchronous, ACID-compliant local database.
- **Cache Partitions:**
  - Content_Store: Blobs, videos, PDFs (high volume, low priority).
  - Action_Queue: User intents, form submissions (low volume, critical priority).
  - Relay_Logs: EORS packet hashes for duplicate suppression.

Cloud Data Structure

- **Relational (PostgreSQL):** User registry, metadata, transactional truth.
- **In-Memory (Redis):** Fast caching for API responses, rate-limiting, and managing active WebSocket connections.
- **Analytics Engine:** Time-series database tracking sync frequencies, EORS hop counts, and regional connectivity mapping to optimize the prediction engine.

8. Security Architecture

- **End-to-End Encryption (E2EE):** EORS packets are impenetrable by middle-nodes.
- **Device Identity Verification:** Every mobile installation generates an ECC key pair. The public key is registered. Cloud endpoints reject payloads signed by unknown keys.
- **Certificate Pinning:** Hardcodes backend TLS certificates into the Flutter app, rendering Man-in-the-Middle (MITM) attacks via rogue local Wi-Fi hotspots impossible.
- **DDoS Protection:** API gateways automatically throttle or drop malformed sync requests.
- **Data at Rest:** The Isar database is encrypted using AES-256-GCM, utilizing a key stored securely in the Android Keystore / iOS Secure Enclave.

9. Deployment Architecture

```
graph LR
  subgraph Edge
    App1[Mobile Device]
    App2[Mobile Device]
  end

  subgraph GCP / AWS
    LB[Global Load Balancer]
    Gateway[API Gateway / WAF]

    subgraph Serverless Compute
      CR1[Cloud Run - Sync Services]
      CR2[Cloud Run - ML/Predict]
      CR3[Cloud Run - EORS Dispatch]
    end

    subgraph Managed Data
      PG[(PostgreSQL Cluster)]
      RD[(Redis Cache)]
      S3[(Object Storage - CDN)]
    end
  end

  App1 --> LB
  App2 --> LB
  LB --> Gateway
  Gateway --> CR1
  Gateway --> CR2
  Gateway --> CR3
  CR1 --> PG
  CR1 --> S3
  CR2 --> RD
  CR3 --> PG
```

- **Cloud Run Containers:** Stateless architecture allows for scaling from 0 to 10,000 instances instantly when a region suddenly regains connectivity and millions of queued events flood the system.
- **Regional Replication:** Deploying server clusters geographically close to target regions (e.g., AWS af-south-1 or ap-south-1) to minimize latency during brief connectivity

windows.

10. Risk Management

Risk Category	Specific Risk	Impact	Deep Mitigation Strategy
Network	Low user density	EORS Relay chain fails to reach a gateway.	Utilize Long-Range (LR) PHY in Bluetooth 5.0; integrate with static community hubs (e.g., local store Wi-Fi).
Storage	Storage overload	App crashes, user uninstalls due to device bloat.	Aggressive compression + semantic LRU eviction. Cap app size via OS-level storage checks.
Hardware	Battery drain	Continuous background scanning depletes battery.	Adaptive scanning: Duty cycle BLE (scan 2s, sleep 8s); fully disable when battery < 15% unless an emergency is triggered locally.
Security	Device compromise	Malicious actor extracts local patient/financial data.	Hardware-backed keystore encryption; local biometrics required to view sensitive cached data.
Data	Split-brain conflicts	Same record edited on two devices offline.	CRDTs with deterministic resolution rules (e.g., "Add wins over Delete",

			"Highest timestamp wins").

11. Deepest Problem-to-Solution Analysis

Educational: Predictive Fetching vs. Stagnant Caching

- *Problem:* Users don't know what they will need to read tomorrow when they lose internet today. Standard caching requires manual "Download for Offline" clicks.
- *Deep Solution:* The system removes cognitive load. The ML model uses Markov Chains to calculate probability matrices of the user's next clicks based on the syllabus. *Result:* By predicting and fetching, we eliminate the "buffering" phase and reduce repeated duplicate downloads across network edges.

Healthcare: Offline Inference vs. Cloud Processing

- *Problem:* Medical AIs require massive compute power typically found only in the cloud, rendering them useless in rural clinics.
- *Deep Solution:* Model Quantization. Converting heavy 32-bit floating-point AI models into 8-bit integer models (using TFLite). This allows advanced symptom classification to run locally on a $50 smartphone processor in milliseconds, ensuring diagnostic safety without dependency.

Communication: Multi-Hop Routing vs. Single Point of Failure

- *Problem:* If the one cellular tower in a valley fails, all 10,000 residents are silenced.
- *Deep Solution:* Decentralization via mesh. By turning every phone into a router, the network heals itself. Priority interrupt logic ensures that if an emergency packet arrives, all other background syncing is paused, guaranteeing ultra-low latency probabilistic delivery.

Essential Services: Idempotent Commits vs. standard REST

- *Problem:* A user submits a payment on a 2G network. The packet reaches the server, money is deducted, but the network drops before the confirmation reaches the phone. The user clicks "Submit" again. They are charged twice.
- *Deep Solution:* The device generates a unique Transaction_ID (UUID). The backend checks PostgreSQL for this UUID. If it exists, it ignores the second request but resends the success confirmation. This guarantees exact-once processing continuity.

12. Glossary

- **AOFIDPS:** Adaptive Offline-First Intelligent Data Pre-Fetching System.
- **EORS:** Emergency Opportunistic Relay System.
- **BLE:** Bluetooth Low Energy.
- **CQRS:** Command Query Responsibility Segregation (Architecture pattern).

- **CRDT:** Conflict-free Replicated Data Type (Algorithm for offline sync).
- **DAG:** Directed Acyclic Graph (Data structure).
- **LRU:** Least Recently Used (Cache eviction policy).
- **PKI:** Public Key Infrastructure (Security framework).
- **TTL:** Time To Live.
- **Idempotency:** A property of certain operations where they can be applied multiple times without changing the result beyond the initial application.

Conclusion

AOFIDPS is not merely an app with an "offline mode"; it is a hyper-resilient, modular, distributed digital platform engineered specifically for the harsh realities of low-connectivity environments. By synthesizing predictive edge caching, lightweight localized AI, secure asynchronous synchronization, and EORS opportunistic mesh networking, the system guarantees uninterrupted access to education, life-saving healthcare, vital communication, and essential economic services independent of continuous telecom infrastructure.